HUSKEY POCKET

Firmware Technical Documentation

Architecture, modules, interface, and features

Development reference • Waveshare ESP32-S3-Touch-LCD-2.8 platform

Contents

Contents 2
1. Overview 3
2. Code Organization 3
3. Application Lifecycle 4
4. vault.h Module — Cryptographic and Data Core 5
5. vault_csv.h and vault_hky.h Modules — Import and Export 7
6. Output Channels to the Host 8
7. Peripherals 9
8. User Interface 9
9. Cross-cutting Security Notes 10

1. Overview

This document describes the internal structure of the Huskey Pocket firmware: how the code is organized, what each module does, and the public interfaces of key functions. It is a development reference that complements the security whitepaper. This document focuses on how the firmware works.

The firmware is an Arduino/ESP-IDF project for the ESP32-S3, written in C++ and organized into headers (.h) by functional area, with a main sketch (.ino) coordinating startup and the main loop. The graphical interface uses LVGL. Encrypted storage uses mbedTLS, NVS, and LittleFS.

Platform and dependencies

Item	Details
MCU / board	ESP32-S3 on Waveshare ESP32-S3-Touch-LCD-2.8
Framework	Arduino-ESP32 (core 3.x)
Graphical interface	LVGL
Cryptography	mbedTLS (AES, PBKDF2, HMAC-SHA)
Storage	NVS (Preferences) + LittleFS
Bluetooth	NimBLE-Arduino 2.x (HID keyboard profile)
USB	TinyUSB (native HID and mass storage)
Sensors	PCF85063ATL RTC, QMI8658C IMU (shared I2C bus)

Key reference constants

Constant	Value and meaning
`PIN_DIGITS`	6 — PIN length
`VAULT_PBKDF2_ITERATIONS`	50,000 — PBKDF2 iterations for device PIN
`HKY_ITERATIONS`	100,000 — PBKDF2 iterations for encrypted export
`VAULT_SALT_LEN`	16 — salt bytes
`VAULT_FIELD_LEN`	64 — maximum length for name, username, password, and secret
`VAULT_NOTES_LEN`	128 — maximum notes length
`VAULT_MAX_ENTRIES`	3,500 — maximum vault entries
`VAULT_MAX_CATS`	16 — maximum categories

2. Code Organization

The code is grouped into four conceptual layers. Individual files may cover more than one layer; the .ino file connects them.

Core and data layer

- password_manager.ino — entry point: setup() and loop(). Selects the startup mode (normal or USB storage), initializes hardware and peripherals, and dispatches background ticks (Snatch Lock and auto-lock).
- vault.h — system core: encryption, key derivation, PIN verification, decoy PIN, FRP, Lost Mode, categories, TOTP, password generation, and the vault read/write API.
- vault_csv.h — CSV import and export (Huskey and browser formats), with an RFC 4180 parser and incremental import.
- vault_hky.h — streaming encrypted .hky import and export using the same CSV parser.

Peripheral layer

- rtc_manager.h — dedicated PCF85063ATL real-time clock.
- snatch_lock.h — QMI8658C IMU-based Snatch Lock.
- sd_manager.h — SD-card mounting, line reading, secure cleanup, and file utilities.
- battery_manager.h, power_manager.h, vibro_manager.h — battery, power/deep sleep, and haptic feedback.
- Display_ST7789.*, Touch_CST328.* — display and touch drivers.

Host input/output layer

- ble_keyboard_manager.h — Bluetooth HID keyboard (NimBLE) and pairing.
- usb_keyboard_manager.h — native USB HID keyboard.
- usb_msc_manager.h — USB mass-storage mode (exposes the SD card to the computer).
- kb_manager.h — abstraction layer that routes typing to the active channel (BLE or USB).

User-interface layer

- pin_screen.h, lock_screen.h, onboarding.h — PIN, lock, and first-run setup screens.
- ui_main.h, ui_list.h, ui_forms.h, ui_settings.h — main screen, entry list, forms, and settings.
- ui_generator.h, ui_qr.h, ui_statusbar.h, power_ui.h — password generator, QR codes, status bar, and power interface.
- custom_kb.h, custom_kb_t9.h — on-screen full and T9 keyboards.
- i18n.h, colors.h, fonts.h — Spanish/English localization, theme palette, and fonts.

3. Application Lifecycle

Startup (setup)

Startup first decides whether to enter USB storage mode. It makes this decision before loading the vault or other features by reading a persistent flag:

```
if (vault_get_usb_msc_mode()) {     // Initialize minimal LVGL and start usb_msc_begin();
return;   // exclusive mode: vault is not loaded }
```

During normal startup, the firmware initializes the display and touch panel, checks Lost Mode, and shows the access screen. After that screen appears, it mounts the microSD card and cleans managed files in the background. The contents of /import are preserved and are not processed at startup. Bluetooth starts after a successful unlock.

Main loop (loop)

The loop services LVGL (interface and touch) and runs background ticks. The Snatch Lock tick is security-relevant: while enabled and the vault is unlocked, it reads the IMU on each loop iteration.

Unlock

After PIN validation, on_unlock() opens the session, starts Bluetooth if BLE keyboard mode is selected, and loads the main screen. The derived key remains in RAM from this point.

Lock

Locking (manual, automatic, Snatch Lock, or re-lock) clears the key from RAM, closes Bluetooth, and returns to the PIN screen or enters deep sleep, depending on the trigger.

4. vault.h Module — Cryptographic and Data Core

This module contains the security model and data access. Its responsibilities include key derivation, PIN verification, field encryption and decryption, decoy vault, FRP, Lost Mode, categories, TOTP, and password generation.

Field encryption

Each sensitive field is encrypted individually with AES-256-CBC. Encryption prepends a random 16-byte IV to the ciphertext and applies PKCS#7 padding. Decryption validates that the length is a multiple of the block size and that padding is valid before accepting the data.

Internal function	Purpose
`_derive_key_into(pin, salt_path, out)`	Derives a 32-byte PBKDF2 key into an explicit buffer without modifying the session key.
`_aes_encrypt / _aes_decrypt`	Encrypts/decrypts a field with the session key, handling the IV and padding.
`_write_control / _verify_control`	Write and verify the encrypted control value used to check whether a PIN is correct without storing the PIN.

Authentication — public API

Function	Description
`bool vault_init(pin)`	Derives both candidate keys (decoy and real) in constant time, opens the corresponding vault, and marks the session active. Records the real-unlock timestamp for Lost Mode.
`void vault_lock()`	Clears the key from RAM and closes the session.
`bool vault_verify_pin(pin)`	Checks a real or decoy PIN without changing the active session; used before sensitive actions.
`bool vault_is_ready()`	Indicates whether a vault is unlocked.
`bool vault_is_decoy()`	Indicates whether the active session is the decoy vault.

Factory Reset Protection (FRP)

Function	Description
`vault_frp_save_pin(pin)`	Saves the FRP PIN with its own salt in a namespace that is not erased by factory reset.
`vault_frp_verify(pin)`	Verifies the FRP PIN.
`vault_frp_get/set/reset_attempts()`	Manage the persistent FRP attempt counter.
`vault_frp_lock_permanent()`	Sets the permanent lock after attempts are exhausted.
`vault_frp_is_locked_permanent()`	Checks the permanent-lock state.

Attempt counter and reset

Function	Description
`vault_get/set/reset_fail_count()`	Manage the persistent failed-PIN counter, reserved before each verification.
`vault_factory_reset()`	Erases the vault, categories, preferences, counters, and decoy remnants; preserves the FRP record and zeros the key.

Decoy vault, Lost Mode, and Snatch Lock

Function	Description
`vault_decoy_set_pin / clear_pin / is_set()`	Set, remove, and check whether a decoy PIN is configured.
`vault_get/set_deadman_days()`	Set Lost Mode duration in days (0 = disabled).
`vault_deadman_check()`	Checks on every boot whether Lost Mode reset is due, with protection against the RTC moving backward.
`vault_get/set_snatch_level()`	Set Snatch Lock sensitivity (0–3).

Vault entries

Function	Description
`vault_save_web(name, user, pass, cat...)`	Create or edit a web entry; safely truncates values to field limits.
`vault_save_secret(name, secret, cat)`	Create or edit a secret entry.
`vault_load(name)`	Load and decrypt an entry by name.
`vault_list(names, count, max, cat)`	List entry names, optionally filtered by category.
`vault_count(cat, type)`	Count entries with optional filters.
`vault_delete(name)`	Delete an entry.
`vault_set_favorite(name, fav)`	Mark or unmark an entry as a favorite.

TOTP and password generation

Function	Description
`vault_totp_code(name, seconds_left)`	Calculate an entry's six-digit TOTP code. The secret and code are not written to diagnostics.
`vault_generate_password(out, len, upper, nums, syms, no_ambig)`	Generate a password using the hardware RNG, unbiased sampling, and a length bounded by the output buffer.
`vault_get/set_gen_config()`	Persist the generator's default settings.

> *The vault uses a hash table on LittleFS with linear probing. Each lookup opens and decrypts records along the probe chain, so performance degrades as the table approaches capacity. This is a known backend trade-off.*

5. vault_csv.h and vault_hky.h Modules — Import and Export

vault_csv.h

Implements CSV import and export. The parser follows RFC 4180 (quoted fields and escaped double quotes) and automatically detects the delimiter and header format (Huskey or browser). Import is incremental: a UI timer processes a few lines per call so the touchscreen remains responsive during large imports.

Function	Description
`vault_export_csv(path)`	Export the vault to a CSV file on the SD card.
`vault_import_csv_begin(path)`	Count lines, detect the format, and prepare incremental import.
`vault_import_csv_step(batch)`	Process up to "batch" lines; stop cleanly if the vault fills up.
`vault_import_csv_progress(...)`	Report progress: imported rows, errors, processed lines, and total.
`vault_import_csv_cancel()`	Cancel an import in progress.

vault_hky.h

Implements the encrypted .hky container. Export derives a key from the export password (separate from the PIN) and writes a format header containing a salt, iteration count, and IV, followed by streamed ciphertext beginning with a fixed verification block. Import decrypts on the fly and feeds the same CSV parser without

ever materializing the complete plaintext CSV. It rejects files whose header specifies fewer iterations than the minimum.

Function	Description
`vault_export_hky(path, password)`	Export the vault encrypted with the given password.
`vault_import_hky_begin(path, password)`	Open and validate the container, then prepare streaming decryption.
`vault_import_hky_cleanup(path)`	Safely clean temporary files after import.

6. Output Channels to the Host

ble_keyboard_manager.h — Bluetooth keyboard

Emulates a BLE HID keyboard using NimBLE. Pairing uses numeric confirmation: a six-digit code is displayed for the user to confirm on screen. Keystrokes are sent only over an encrypted and authenticated link.

Function	Description
`ble_kb_begin() / ble_kb_end()`	Start and stop the BLE stack and advertising.
`ble_kb_connected()`	Indicates whether a physical connection is active.
`ble_kb_secure()`	Indicates that the link is encrypted and authenticated; required before typing.
`ble_type_string / ble_type_user / ble_type_pass / ble_type_secret`	Type strings or specific fields to the host.
`ble_type_tab() / ble_type_enter()`	Send Tab and Enter.
`ble_kb_num_bonds()`	Return the number of saved bonds.
`ble_kb_delete_all_bonds()`	Delete all bonds from the stack and storage.

usb_keyboard_manager.h — USB keyboard

Emulates a native USB HID keyboard using TinyUSB. Provides an interface similar to BLE (usb_type_string, usb_type_user, usb_type_pass, usb_type_secret, usb_type_tab, usb_type_enter). The HID descriptor is passed to the framework with its declared size.

kb_manager.h — routing

A thin layer that routes typing functions to the active channel (BLE or USB) according to the user preference, so the rest of the interface does not need to know which transport is in use.

usb_msc_manager.h — mass storage

Exposes the SD card to the host as a USB drive. This is an exclusive mode and does not run alongside the vault or keyboards. Block access validates ranges against card capacity and rejects malformed transfers; optional read-only mode is available. The vault partition is on internal Flash and is never exposed.

Function	Description
`usb_msc_begin()`	Start mass-storage mode and show the minimal screen.
`vault_get/set_usb_msc_mode()`	Read and set the persistent flag that enables this mode at boot.
`vault_get/set_usb_msc_readonly()`	Read and set the read-only preference.

7. Peripherals

rtc_manager.h — real-time clock

Controls the PCF85063ATL RTC over I2C. An external backup battery is required to retain time when main power is removed. This is the vault's only time source, used by TOTP and Lost Mode. Writes validate all ranges; reads mask status bits.

Function	Description
`rtc_begin()`	Initialize the chip, clear the stop bit, and detect time loss.
`rtc_set(year, month, day, hour, min, sec)`	Set the time after validating each range.
`rtc_is_set()`	Indicate whether the RTC has a valid time.
`rtc_get_time_str / rtc_get_date_str`	Return formatted time and date strings.
`rtc_get_epoch()`	Return a timestamp for vault calculations.

snatch_lock.h — Snatch Lock

Reads the QMI8658C IMU in software on each loop iteration. It calculates combined acceleration and rotation magnitudes; if both exceed the selected level's threshold at the same time, it enters deep sleep. Requiring both conditions reduces false positives. It does not act during a sensitive operation.

Function	Description
`snatch_lock_init()`	Configure the IMU (ranges and sample rate) and check that it is present.
`snatch_lock_tick()`	Called on each loop iteration; checks for a snatch when enabled and the vault is unlocked.

sd_manager.h — SD card

Mounts the SD card and provides file utilities: line reading (with controlled discard of oversized lines), secure file cleanup, and creation of import/export directories. Managed-file cleanup overwrites contents before deletion, subject to Flash and filesystem limitations.

power_manager.h, battery_manager.h, vibro_manager.h

- power_manager.h — power management and deep-sleep entry/exit, turning off the display, touch panel, and SD card before sleep.
- battery_manager.h — battery-level reading.
- vibro_manager.h — haptic feedback (vibration) during interactions.

8. User Interface

The interface is built with LVGL. It is organized into screens, each in its own file, plus shared components such as on-screen keyboards, status bar, themes, and text.

Access screens

Module	Purpose
onboarding.h	First-run setup: set the PIN, date/time, and initial preferences.
pin_screen.h	PIN screen: digit entry, attempt handling, FRP flow, and decoy timer.
lock_screen.h	Lock screen and transition to the PIN screen.

Main screens

Module	Purpose
ui_main.h	Main screen after unlock; navigation and shortcuts.
ui_list.h	Entry list: search, category filtering, multi-select, details, and entry actions.
ui_forms.h	Entry creation and editing forms.
ui_settings.h	Settings for security, keyboard, language, theme, USB/BLE modes, import/export, Lost Mode, and Snatch Lock.

Components and utilities

Module	Purpose
ui_generator.h	Password generator with length and character-set options.
ui_qr.h	Display data as a QR code.
ui_statusbar.h	Status bar (time, battery, indicators).
power_ui.h	Power and shutdown interface.
custom_kb.h / custom_kb_t9.h	On-screen keyboards: full and T9.
i18n.h	ES/EN localization through key-indexed tables.
colors.h / fonts.h	Theme palette and font definitions.

9. Cross-cutting Security Notes

Some security invariants span multiple modules rather than living in a single one. Keep these in mind when changing any part of the code:

- The PIN-derived key exists only in RAM and is explicitly cleared on every lock and reset. Any new session-opening or session-closing path must preserve this cleanup.
- The decoy vault must never persist real data: save functions check decoy mode and simulate success without writing. New write functions must do the same.

- Sensitive material (PINs, passwords, secrets, keys, TOTP codes) must never be logged to the serial port or another diagnostic channel.
- Real and decoy PIN verification should take constant time relative to one another to avoid revealing which was entered.
- PIN and FRP attempt counters must be persistently reserved before verification, not after.
- All external input (imported files, USB/BLE data) is untrusted and must be validated for size and range before use.

> *Secure Boot is enabled on the author's unit. Huskey Pocket has not received a security audit. Memory reviews and functional tests do not certify device security.*